# `brain/carl_endocrine.py` — Full Line-by-Line Walkthrough

**Length:** 188 lines. **Role:** simulates four brain hormones (dopamine, norepinephrine, serotonin, acetylcholine) as simple decaying numbers, and uses their levels to modulate behavior elsewhere in the system. Wired into `carl_simulation.py` as `self.endocrine`.

---

## Lines 1–5: Header and import

```python
"""
carl_endocrine.py — Neuromodulatory Endocrine Kinetics (END-201..204)
Integrated into unified SoC state slices.
"""
import numpy as np
```

Only dependency is NumPy, used just for `np.clip()` — this file has no neural network, no physics, nothing exotic. It's pure arithmetic simulating hormone-like decay curves.

## Lines 8–25: The tunable constants

```python
class EndocrineSystem:
    BASELINE_DA  = 1.0
    BASELINE_NE  = 0.1
    BASELINE_SHT = 0.8
    BASELINE_ACH = 0.5
```

Each hormone has a **resting value** it drifts back to when nothing's happening — dopamine's resting level (1.0) is deliberately higher than norepinephrine's (0.1), meaning CARL's default state is "calm/moderately content" (DA near its comfortable middle) rather than "alert" (NE stays low unless something triggers it).

```python
    DECAY_DA  = 2.0
    DECAY_NE  = 3.0
    DECAY_SHT = 1.0
    DECAY_ACH = 2.5
```

These are **decay rates** — how fast each hormone snaps back to baseline after a spike. Higher number = faster return to normal. NE has the fastest decay (3.0) — alertness spikes are meant to be short-lived, matching how real adrenaline/norepinephrine responses spike and fade quickly. Serotonin has the slowest decay (1.0) — calm/satiation is meant to linger longer, which matches its real biological role as a slower-acting mood modulator.

```python
PI_0      = 1.0
GAMMA_ACH = 0.6
GAMMA_NE  = 0.4
GAMMA_SHT = 0.3
```

`PI_0` is the baseline precision weight (used below in `get_precision_weight`). The three `GAMMA_*` values are weighting coefficients for how strongly each hormone pushes that precision weight up or down — acetylcholine has the strongest influence (0.6), matching its docstring-stated role as the primary "sharpen attention" hormone.

```python
MAX_NM = 3.0
MIN_NM = 0.0
```

Hard ceiling and floor every hormone gets clamped to after every update — no hormone can go negative or run away to infinity.

### Lines 27–34: `__init__`

```python
def __init__(self):
    self.DA  = self.BASELINE_DA
    self.NE  = self.BASELINE_NE
    self.SHT = self.BASELINE_SHT
    self.ACh = self.BASELINE_ACH
    self.reward_avg = 0.0
    self.reward_avg_alpha = 0.01
    self.last_surge_event = ""
```

Every hormone starts at its resting baseline — CARL "wakes up" calm, not stressed or overstimulated. `reward_avg` starts at zero and will track a **running average of dopamine over time** (used in `get_reward_prediction_error` below) — `reward_avg_alpha = 0.01` is how fast that running average adapts (very slowly — it takes roughly 100 ticks, over 3 seconds, for the average to meaningfully shift, meaning it represents "typical recent

reward level," not "reward right this instant"). `last_surge_event` is a text label (empty string = nothing happened) used purely for the telemetry log and console diagnostics in `carl_simulation.py`.

### Lines 36–54: `step()` — the decay physics

```python
    def step(self, dt: float = 0.033):
        """Advance all neuromodulator levels by one timestep using first-orde
        ...
            dX/dt = -DECAY_X * (X - BASELINE_X)
        ...
        """
        self.DA  += -self.DECAY_DA  * (self.DA  - self.BASELINE_DA)  * dt
        self.NE  += -self.DECAY_NE  * (self.NE  - self.BASELINE_NE)  * dt
        self.SHT += -self.DECAY_SHT * (self.SHT - self.BASELINE_SHT) * dt
        self.ACh += -self.DECAY_ACH * (self.ACh - self.BASELINE_ACH) * dt
```

This is **exponential decay toward a setpoint**, the same math that describes a hot cup of coffee cooling toward room temperature, or a capacitor discharging toward zero voltage. The formula `dX/dt = -k(X - baseline)` says: "the further you are from baseline, the faster you move back toward it" — a spike of dopamine to 2.5 (baseline 1.0) will initially fall fast, then slow down as it approaches 1.0, asymptotically settling rather than overshooting or oscillating. This is a first-order linear ODE, solved here using simple **Euler integration** (current value + rate × timestep) rather than an exact closed-form solution — fine at this small a timestep (`dt=0.033`), and simpler to write/reason about.

```python
        self.DA  = np.clip(self.DA,  self.MIN_NM, self.MAX_NM)
        self.NE  = np.clip(self.NE,  self.MIN_NM, self.MAX_NM)
        self.SHT = np.clip(self.SHT, self.MIN_NM, self.MAX_NM)
        self.ACh = np.clip(self.ACh, self.MIN_NM, self.MAX_NM)
```

Re-clamps every hormone into `[0.0, 3.0]` after the decay update — a defensive safety net in case a large surge (see below) pushed a value past the ceiling before decay had a chance to pull it back down.

### Lines 56–98: The four `surge_*` methods

```python
```

```
    def surge_da(self, amount: float = 0.35):
        self.DA += amount
        self.last_surge_event = f"DA+{amount:.2f}"
```

All four surge methods (`surge_da`, `surge_ne`, `surge_sht`, `surge_ach`) follow the exact same pattern: **add** a fixed amount to the current level (not set it to a fixed value — meaning two surges in quick succession stack), and record a text label for logging. They don't clamp immediately (the docstring notes "clamping to MAX_NM is applied on the next `step()`") — meaning for a fraction of a tick, a hormone value can technically sit above 3.0 before the next `step()` call clips it back down. Harmless in practice since nothing reads these values in that narrow window before `step()` runs (checked in `carl_simulation.py`: surges happen, then `endocrine.step()` is called immediately after, all within the same tick).

Each surge is called from a different real-world trigger in `carl_simulation.py`: `surge_da(0.7)` on eating food (line 614), `surge_ne(...)` on obstacle proximity/boundary breach/hazard proximity (multiple call sites, different amounts per severity), `surge_sht(0.3)` also on eating (satiation follows reward). Note: **`surge_ach()` is defined here but I didn't find it actually called anywhere in `carl_simulation.py`** — acetylcholine only ever moves via its baseline decay in `step()`, meaning the "novel environments trigger attention" behavior described in its own docstring isn't currently wired to any actual novelty-detection event. This is a small but real gap: the mechanism exists, but nothing in the running loop pulls the trigger on it yet.

### Lines 100–117: `get_precision_weight()` — the attention dial

python

```
    def get_precision_weight(self) -> float:
        pi = self.PI_0 * (
            1.0
            + self.GAMMA_ACH * self.ACh
            + self.GAMMA_NE  * self.NE
            - self.GAMMA_SHT * self.SHT
        )
        return max(0.1, pi)
```

This produces a single scalar multiplied against the entire 37-dimensional sensory input in `carl_simulation.py`'s `step()` method (`u_weighted = pi_sensory * u_raw`). The formula: start from 1.0 (neutral), then acetylcholine and norepinephrine each *add* to it (both hormones associated with alertness/attention in real neuroscience push CARL to trust its senses more), while serotonin *subtracts* from it (a calm, satiated state relies more on internal expectation/priors than moment-to-moment sensory data — this is a genuine

nod to Bayesian brain / predictive-processing theory, where "precision" is the technical term for how much weight is given to sensory prediction error versus prior belief). Floored at 0.1 so CARL is never fully sensory-blind even in the calmest state.

### Lines 119–129: `get_exploration_noise_scale()`

```python
    def get_exploration_noise_scale(self) -> float:
        return max(0.01, self.NE)
```

Directly returns the current norepinephrine level (floored at 0.01) as a noise multiplier — this value gets passed into the reservoir's `step()` in `carl_simulation.py` (`noise_scale = self.endocrine.get_exploration_noise_scale()`), meaning **higher stress/ alertness literally makes CARL's motor output noisier/more randomly exploratory**. The docstring's biological framing (locus coeruleus, the brain region that produces norepinephrine, gating between exploit/explore modes) is a real, cited neuroscience concept, not just a metaphor invented for this project.

### Lines 131–147: `get_reward_prediction_error()`

```python
    def get_reward_prediction_error(self) -> float:
        rpe = self.DA - self.reward_avg
        self.reward_avg += self.reward_avg_alpha * (self.DA - self.reward_avg
        return rpe
```

Computes **Reward Prediction Error**, the actual technical term from dopaminergic neuroscience (Schultz's classic RPE finding — dopamine neurons don't fire for reward itself, they fire for reward *better than expected*). `self.reward_avg` acts as "expectation," slowly tracking DA's recent history at the `alpha=0.01` rate set in `__init__`. If current DA is above that slow-moving average, RPE is positive (things are going better than the recent norm); below it, negative. **Important:** this method has a side effect (it updates `reward_avg` every time it's called) baked into what looks like a pure getter — calling this method twice in a row would update the average twice, which could cause subtle bugs if called from more than one place per tick. Worth checking whether `carl_simulation.py` actually calls this anywhere — a quick grep shows **it currently doesn't** — this method is fully built but not yet wired into the main control loop at all, another concrete "built but standalone" piece consistent with the project's honest self-assessment.

### Lines 149–161: `get_behavioral_mode()` — the mode switch

```python
    def get_behavioral_mode(self, battery_level: float, food_visible: bool) -
        if food_visible:
            return "EXPLOIT"
        elif battery_level < 0.15 or self.NE > 2.0:
            return "SURVIVE"
        else:
            return "EXPLORE"
```

This is a simple priority-ordered decision tree, not a learned policy — plain `if/elif/else`. Food visibility always wins first (if you can see food, go get it, regardless of anything else). Otherwise, critical battery (<15%) *or* extreme stress (NE > 2.0, well above its normal 0.1 baseline) forces survival mode. Everything else defaults to explore. Notice NE alone can force SURVIVE mode even with a full battery — meaning a sufficiently frightening situation (repeated close obstacle encounters) can push CARL into "desperate" behavior independent of hunger, which is a deliberate cross-wiring between the threat-response and survival-mode systems.

### Lines 163–170: `get_hunger_drive()`

```python
    def get_hunger_drive(self, battery_level: float) -> float:
        return float(np.clip(1.0 - battery_level, 0.0, 1.0))
```

The simplest method in the file: hunger is just the inverse of battery level, clamped to [0, 1]. Full battery (1.0) = zero hunger. Empty battery (0.0) = maximum hunger (1.0). The docstring's point about "animals don't suddenly start navigating toward food at a threshold" is the actual design philosophy behind using a *continuous* value here rather than a boolean — it's what lets `get_replay_blend_weight()` below produce a smooth blend instead of a jarring on/off switch.

### Lines 172–182: `get_replay_blend_weight()`

```python
    def get_replay_blend_weight(self, battery_level: float, max_nav_value: fl
        hunger = self.get_hunger_drive(battery_level)
        signal_strength = float(np.clip(max_nav_value / 2.0, 0.0, 1.0))
        return float(np.clip(signal_strength * (0.3 + 0.7 * hunger), 0.0, 1.0
```

This is the exact function `carl_simulation.py` calls to decide how much the hippocampal "replay navigation" signal should influence movement. `signal_strength` normalizes the strongest currently-known reward-graph value (`max_nav_value`, arbitrary scale, assumed to typically top out around 2.0) into a 0–1 fraction. The blend formula `signal_strength × (0.3

- 0.7 × hunger)`means: even at zero hunger (full battery), a strong nav signal still gets a minimum 30% weight (`0.3` floor term) — CARL will still mildly drift toward known-good areas even when it's not desperate — and that influence scales up to 100% weight as hunger approaches 1.0. This is the concrete mechanism behind the docstring's claim: "even at full battery, if there's a strong nav gradient, follow it mildly."

### Lines 184–187: `__repr__`

```python
    def __repr__(self):
        return (f"DA={self.DA:.2f}, NE={self.NE:.2f}, "
                f"5HT={self.5HT:.2f}, ACh={self.ACh:.2f}, "
                f"pi={self.get_precision_weight():.2f}")
```

Defines what `print(self.endocrine)` or an f-string like `f"{self.endocrine}"` actually displays — this is exactly what shows up in `carl_simulation.py`'s once-per-second console diagnostic line. Calling `get_precision_weight()` inside `__repr__` means printing the object has a (harmless, since that method is a pure calculation with no side effects) extra computation baked into what's normally expected to be a cheap debug-string operation — not a problem here, just worth knowing `repr()` isn't "free" on this class.

---

## Summary

This file is small, clean, and does exactly one job: simulate four decaying scalar values and expose several derived quantities (precision weight, noise scale, behavioral mode, hunger, replay blend) computed from them. It's the most "just math" file in the brain/ folder — no neural network, no learned parameters, entirely hand-designed kinetics and thresholds. Two things worth knowing for questions: `surge_ach()` exists but nothing currently calls it, and `get_reward_prediction_error()` exists but isn't called anywhere in the main loop either — both are complete, tested (the new test suite covers this file thoroughly), but standalone capabilities waiting for a caller.